# Locality-based moral planning with linear temporal logic values

ADRIAN SATJA KURDIJA*, Faculty of Electrical Engineering and Computing, University of Zagreb, Croatia
MATEJ MIHELČIĆ, Department of Mathematics, Faculty of Science, University of Zagreb, Croatia and Department of Electronics, Ruđer Bošković Institute, Croatia

**Objective:** We study the CONFLICT problem in logic-based ethical planning with Linear Temporal Logic (LTL) values, which is PSPACE-complete in general, and target a tractable fragment relevant to structured robotic domains.

**Method:** We introduce locality assumptions over propositions and actions: most propositions are indexed along an ordered local axis, each action depends on and modifies only a bounded interval, the number of global propositions is constant, and locality also constrains feasible temporal action order. Under these assumptions, we derive a polynomial-time locality-based planning algorithm.

**Results:** We evaluate against brute-force and automata/progression baselines on a deterministic 110-instance benchmark suite spanning three modeled robotic families (linear traffic lights, charging extension, and narrow three-lane corridor). With a common 15-second timeout and independent validation, the locality planner solves 110/110 instances, while automata/progression solves 105/110 and brute force 90/110. The locality method also has the lowest runtime distribution and significantly smaller per-instance runtimes in one-sided Mann–Whitney U tests.

**Conclusion:** Locality-aware state compression yields a practically robust and fast exact planner in high-locality domains, while retaining expressive temporal value constraints.

**JAIR Associate Editor:** Insert JAIR AE Name

**JAIR Reference Format:**
Adrian Satja Kurdija and Matej Mihelčić. 2026. Locality-based moral planning with linear temporal logic values. *Journal of Artificial Intelligence Research* 1 (March 2026), 20 pages. DOI: 10.1613/jair.1.xxxxx

## 1 Introduction

Planning is crucial for autonomous systems like robotics. A framework for logic-based ethical planning with intended application to robotics was introduced by Grandi et al. (**grandi2022logic**). For a key problem, which is to ensure that actions of an agent align with multiple predefined (moral or other) values, linear temporal logic is used to represent values and determine feasible action plans. Evaluation criteria for individual plans were proposed in (**grandi2023moral**). General propositional planning problems (ie, PLANSAT) are known to be PSPACE-complete (**BYLANDER1994165**), as well as the problem of moral conflict as defined in Grandi et al. (**grandi2022logic**). In this work we introduce locality-based constraints of actions and propositions to reduce the search space and computational complexity.

Linear Temporal Logic (LTL) enables specifying constraints over sequences of actions (**10.5555/975331**). A set of moral values $\Omega$ consists of LTL formulas (**4567924**) that describe temporal properties of atomic propositions (evaluated under some history $H$) using operators such as negation ($\neg\phi$), conjunction ($\phi_1 \wedge \phi_2$), "next" ($X\phi$),

*Corresponding Author.

Authors' Contact Information: Adrian Satja Kurdija, ORCID: 0000-0003-2313-0396, adrian.kurdija@fer.unizg.hr, Faculty of Electrical Engineering and Computing, University of Zagreb, Zagreb, Croatia; Matej Mihelčić, ORCID: 0000-0002-1023-8413, matmih@math.hr, Department of Mathematics, Faculty of Science, University of Zagreb , Zagreb, Croatia, matej.mihelcic@irb.hr and Department of Electronics, Ruđer Bošković Institute , Zagreb, Croatia.

This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2025 Copyright held by the owner/author(s).
DOI: 10.1613/jair.1.xxxxx

"until" ($\phi_1 \mathsf{U} \phi_2$), "henceforth" ($G\phi$), and "eventually" ($F\phi$). For a given moral problem consisting of moral values $\Omega$, action theory $\gamma$, and an initial state $s_0$, the CONFLICT problem asks if there is an action plan $\pi$ satisfying all values from $\Omega$. If there is no such action plan, some values have to be broken, so tradeoffs between values have to be accounted for, which explains the name *moral conflict*. However, we note that the concerned values do not have to be ethical/moral in the strict sense of the word. We decided to follow the previous terminology while keeping in mind the generality of the approach.

Grandi et al. (**grandi2022logic**) have shown that several planning tasks are intractable (PSPACE-complete). Namely, CONFLICT is shown to be PSPACE-complete using reduction from the propositional STRIPS planning problem called PLANSAT (**BYLANDER1994165**). The difficulty of the problem arises from the exponential number of possible plans. Using additional assumptions at the cost of generality, we define a constrained version of the CONFLICT problem and present the algorithm to solve it. Namely, to enable polynomial complexity, we restrict actions to affect only a local set of propositions, introducing the *locality* assumption.

The proposed framework allows modelling many real-world scenarios involving planning. One example includes utilizing propositions to express properties involving linear space order. Many robots involved in repairs or manufacturing adhere to such constraints (e.g. robots used to pick and pack food in the factories, gantry robots used in automotive industry, robots dispatching pallets in the warehouses, robots in electronic chip manufacturing or pharmaceutical production). Utilizing proposed logic, these robots could move back and forth satisfying expressive, complex constraints. One important constraint is not to harm human workers that might cross the path of the traversing robots. This shows the importance of having the possibility to add moral constraints in the planning process to create safe, flexible and efficient robots. To complement the theoretical analysis, we perform an empirical comparison against brute-force and automata/progression baselines on systematic instances derived from the three examples in this paper, reporting both runtime and completion behavior under equal timeout budgets.

The paper is organized as follows. Sect. 2 describes the related work. In Sect. 3 we describe the framework and the proposed algorithm. In Sect. 4 we present examples. Sect. 5 reports benchmark-based evaluation. Conclusions are given in Sect. 6.

## 2 Related Work

Logic-based temporal planning has previously been studied in the context of robotics and autonomous systems. Various approaches have been used to address uncertainty, efficiency, and fairness in decision-making. Significant focus has been on using LTL, as well as probabilistic temporal logic (**konur2011realtimeprobabilistictemporallogics**), to define and schedule complex tasks in dynamic and uncertain environments. Earlier work such as (**fainekos2009temporal**) investigated temporal logic motion planning for dynamic robots, formulating a hierarchical approach that integrates motion tracking and formal logic specifications to satisfy high-level tasks. Research (**cai2024reinforcement**) explored methods based on reinforcement learning (RL) to satisfy soft temporal logic constraints in environments where full compliance is not feasible due to motion uncertainties and environmental uncertainties. Some works have investigated probabilistic and reactive planning frameworks. Yoo et al. (**yoo2013probabilistic**) introduced a probabilistic computation tree logic (PCTL) extension to incorporate resource constraints in motion planning, while Cai et al. (**cai2021probabilistic**) developed a probabilistic LTL motion planning strategy that takes into account both environmental uncertainties and motion uncertainties. In contrast, Zhou et al. (**kan2024local**) proposed local observation-based reactive temporal logic planning, focusing on real-time human-robot interactions.

While heuristic search and approximate methods—such as those utilizing reinforcement learning to satisfy soft temporal logic constraints or probabilistic LTL frameworks—offer scalability in uncertain or highly unconstrained environments, they often inherently sacrifice strict guarantees of value satisfaction. In safety-critical robotic applications, such as robots performing tasks in the presence of human workers, approximate compliance is simply

insufficient because a potential collision could cause serious injury or fatality. Our approach contrasts directly with these heuristic methods by providing an exact planning algorithm. By exploiting the inherent structural locality of specific operational domains, we achieve the computational efficiency typically associated with approximate methods—specifically, polynomial-time solvability—without relinquishing the absolute mathematical guarantees required for strict moral and safety compliance.

A related research area has been automated reasoning and decision-making for autonomous systems. Meli et al. (**melilogic**) have employed logic programming to improve deliberative robotic task planning, allowing robots to efficiently evaluate and execute high-level actions. Meanwhile, Shea-Blymyer and Abbas (**10.1145/3460975**) have formalized *algorithmic ethics* for autonomous systems, introducing a framework based on deontic logic (**von1951deontic**) to satisfy regulatory and ethical constraints. Similarly, Liang and Vasile (**fairplanning**) addressed concerns about fairness in Mobility-on-Demand systems by using fairness metrics based on temporal logic in decision-making algorithms. In recent work, the use of automata-based approaches has appeared, such as Toscano-Moreno et al. (**li2024spin**) which proposed SPIN-based LTL path planning, utilizing model checking for efficient motion planning in environments with uneven terrain. Cai, Liu, Zhou, Ma, Zhao, Wu and Ma (**smith2024driving**) proposed retrieval-augmented reasoning with Large Language Models (LLMs) to improve the interpretability of autonomous vehicle decision making.

A particularly relevant line of work for finite-trace temporal planning is automata/product-state search for $LTL_f$ formulas (**degiacomo2013ltlf**; **camacho2018finite**). Intuitively, the planner keeps two pieces of information at each step: the current world state, and a compact summary of "what still needs to happen" for the temporal goals to be satisfied. After every action, this temporal summary is updated; this update is often called progression. Search is then performed over pairs of the form (world state, temporal summary), so two identical world states are treated differently if they have different remaining temporal obligations. The method is general and theoretically clean, but the number of such pairs can grow quickly. Our automata/progression baseline in Section 5 follows this paradigm, while our main method additionally exploits interval locality to keep the explored state space smaller.

## 3 Locality-Based Planning

Section 3.1 describes the relevant part of the framework defined in (**grandi2022logic**), Section 3.2 describes our additional assumptions and Section 3.3 describes the proposed algorithm.

### 3.1 Logic-Based Ethical Planning

The world is described by atomic propositions $Prop = \{p_1, \dots, p_n\}$. Their Boolean truth values are represented as a state $s \in 2^{Prop}$ which consists of all propositions that are true at a given moment.

Linear Temporal Logic over Finite Traces, noted as $LTL_f$, is a temporal logic used to represent an agent's values and goals over finite traces. $LTL_f$ includes the operators from classical linear temporal logic such as "next" ($X$) and "until" ($U$), allowing for expressions about the progression of time and the satisfaction of certain conditions. Additionally, the standard operators "henceforth" ($G$) and "eventually" ($F$) are defined as $G\varphi := \neg(\top U \neg\varphi)$ and $F\varphi := \top U \varphi$. Formally, the grammar of $LTL_f$ is defined as follows ($p \in Prop$):

$$\varphi ::= p \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid X\varphi \mid \varphi_1 U \varphi_2 \mid G\varphi \mid F\varphi. \tag{1}$$

In this paper, we expand the language with additional logical operators in $LTL_f$, defined as follows:

$$\varphi \vee \psi \equiv \neg(\neg\varphi \wedge \neg\psi), \quad \varphi \rightarrow \psi \equiv \neg\varphi \vee \psi. \tag{2}$$

There is a set $Act$ of *actions* and each action $a \in Act$ can change values of some propositions according to the corresponding preconditions. Formally, we define *action theory* as a pair of functions

$$\gamma^+, \gamma^- : Act \times Prop \rightarrow \mathcal{L}_{PL}, \tag{3}$$

where $\mathcal{L}_{PL}$ is the propositional logic fragment of $LTL_f$. These functions represent preconditions for the actions. Namely, after action $a$, proposition $p$ becomes true if the precondition $\gamma^+(a,p)$ is true, and false if the precondition $\gamma^-(a,p)$ is true. These rules will be formalized in Eq. (4).

A sequence of actions $\pi = (a_1, \ldots, a_k)$ is called *action plan*. In the formal problem definition, there is no a priori bound on $k$: satisfiability is defined over all finite plans.

The described rules of action theory allow us to define a *history* $H(\pi, s_0, \gamma)$ generated by an action plan $\pi$ from an initial state $s_0$ under action theory $\gamma$. Namely, $H(\pi, s_0, \gamma)$ is a sequence of states $H_0, H_1, H_2, \ldots, H_k \in 2^{Prop}$ with $H_0 = s_0$. To define how $H_t$ transforms into $H_{t+1}$, we first define the $\models$ relation: we say that $H, t \models \varphi$ if the formula $\varphi \in LTL_f$ holds at time point $t$ in the history $H$. More specifically,

- $H, t \models p \iff p \in H_t$ for atomic propositions $p \in Prop$.
- $H, t \models \neg\varphi \iff H, t \not\models \varphi$.
- $H, t \models \varphi_1 \wedge \varphi_2 \iff H, t \models \varphi_1$ and $H, t \models \varphi_2$.
- $H, t \models X\varphi \iff H, t+1 \models \varphi$ (ie, $\varphi$ holds in the next state).
- $H, t \models \varphi_1 U \varphi_2 \iff$ there exists a time $t' \geq t$ such that $H, t' \models \varphi_2$ and for all $t''$ with $t \leq t'' < t'$ it holds that $H, t'' \models \varphi_1$ (ie, $\varphi_1$ holds until $\varphi_2$ becomes true).
- $H, t \models F\varphi \iff$ there exists a time $t' \geq t$ such that $H, t' \models \varphi$ (ie, $\varphi$ holds at some future point, *eventually*).
- $H, t \models G\varphi \iff$ for all $t' \geq t$ it holds that $H, t' \models \varphi$ (ie, $\varphi$ holds at every point in the future, *henceforth*).

Now, for each action $a_t \in \pi$ and corresponding time $t$, the next state $H_{t+1}$ is determined according to the action theory $\gamma$ by updating the propositions in $H_t$ based on the effects of the action $a_t$:

$$H_{t+1} = \Big(H_t \setminus \{p \in Prop : H_t \models \gamma^-(a_t, p)\}\Big) \cup \{p \in Prop : H_t \models \gamma^+(a_t, p)\}. \tag{4}$$

Here, for each proposition $p$, we keep or change its truth value based on $\gamma^+(a_t, p)$ and $\gamma^-(a_t, p)$, the positive and negative effect preconditions associated with action $a_t$.

A set of moral values is given by $\Omega \subseteq LTL_f$. We say that a given moral problem, defined as a triple $\mathcal{M} = (\Omega, \gamma, s_0)$, is a moral conflict if there is no satisfactory action plan. Formally, $\mathcal{M} \in CONFLICT$ if there is no action plan $\pi$ such that the history $H(\pi, s_0, \gamma)$ satisfies all values from $\Omega$. Formally, an action plan $\pi$ is satisfactory if

$$\forall \varphi \in \Omega, \;\; H(\pi, s_0, \gamma), 0 \models \varphi. \tag{5}$$

For example,

- $H, 0 \models p$ when $p \in s_0$,
- $H, 0 \models Fp$ when $p$ is *eventually* satisfied,
- $H, 0 \models Gp$ when $p$ is *always* satisfied,
- $H, 0 \models FGp$ when $p$ is *ultimately* satisfied, ie, it holds in the last state.

If there is no such action plan, some values have to be broken, so tradeoffs between values have to be accounted for, which explains the name *moral conflict*.

Grandi et al. (**grandi2022logic**) showed that CONFLICT is PSPACE-complete using reduction from the propositional STRIPS planning problem called PLANSAT (**BYLANDER1994165**). This means that in the general case, the space of all action plans cannot be efficiently searched in order to find a plan satisfying $\Omega$.

### 3.2 Locality-Based Moral Conflict

Our contribution lies in restricting CONFLICT to a polynomially solvable variant using additional assumptions that reduce its generality without sacrificing too much of its practical use. Motivated from various real-world examples concerning planning in what is essentially a 1D or 2D space (roads, warehouses), such as the examples

in Sect. 4, we assume that most propositions – all except for a constant number of them – are localized. Formally, we introduce locality as the mapping

$$loc : Prop \longrightarrow \{0, 1, \ldots, n\}, \tag{6}$$

where locations $1, \ldots, n$ represent positions of the *local* propositions ($loc(p) > 0$), while the propositions with $loc(p) = 0$ are considered *global*. To limit the search space, we posit that there are at most $M$ global propositions and at most $M$ propositions at each position:

$$\forall i \in \{0, 1, \ldots, n\}, \quad |p \in Prop \ : \ loc(p) = i| \leq M. \tag{7}$$

Each action $a$ depends on, and changes, only the propositions from an interval of at most $L$ locations, along with any global propositions. This local interval depends on the action. To formalize this constraint, we first define $\mathcal{L}_{PL}^{i,j}$ as a fragment of $\mathcal{L}_{PL}$ containing only propositions $p \in Prop$ such that $loc(p) = 0$ or $i \leq loc(p) \leq j$. Now, for each action $a \in Act$, we assume that there are natural constants $l(a)$ and $r(a)$, representing the left and right action boundaries, such that:

$$\begin{aligned} 1 \leq l(a) \leq r(a) \leq n, \\ r(a) - l(a) < L, \\ \gamma^+(a, p_i),\ \gamma^-(a, p_i) \in \mathcal{L}_{PL}^{l(a),r(a)}, \quad \forall i \in [l(a), r(a)], \\ \gamma^+(a, p_i) = \gamma^-(a, p_i) = \bot, \quad \forall i \in [1, n] \setminus [l(a), r(a)]. \end{aligned} \tag{8}$$

Therefore, only the propositions from $\mathcal{L}_{PL}^{l(a),r(a)}$ can be changed by or participate in the preconditions for action $a$.

Additionally, we assume that the local ordering of propositions constrains the temporal order of actions. Informally, the temporal order of actions is roughly left-to-right with allowed deviations. If the actions' intervals are sufficiently apart, the action on the first interval must be performed before the action on the second. Formally, if $r(b) - l(a) \geq 2L$, then action $a$ cannot be performed after action $b$. A larger value of $L$ permits more flexibility in the order of actions. In terms of the action plan $\pi = (a_1, \ldots, a_k)$ this condition can be written as

$$r(a_j) - l(a_i) \geq 2L \implies i < j. \tag{9}$$

The final constraint is that the value set $\Omega$ contains only formulas with at most one of the operators $F$, $G$, $FG$, or $U$, operating on a propositional logic formula containing at most $K$ atomic propositions, either all local, or all global. In the local case, the absolute difference between their indices can be at most $L$.

The integers $K$, $L$, $M$ are introduced as fixed constants to limit the search space, and therefore the algorithm complexity. As a reminder, $K$ is the maximum number of propositions in each value formula, $L$ is the maximum number of local propositions each action can influence, and $M$ is the maximum number of global propositions and propositions at each location. For simplicity of complexity analysis, we will assume that $K, L = O(M)$.

### 3.3 Proposed Algorithm

Under the above assumptions, CONFLICT can be solved in polynomial time. The basic idea stems from the temporal constraint:

$$r(b) - l(a) \geq 2L \implies a \text{ precedes } b,$$

which means that if action $a$ uses/changes $p_i$, this action precedes all actions $b$ such that $r(b) \geq i + 2L$. Thus, while performing an action plan, we do not need to keep track of "distant" propositions. We only need to keep track of the truth values of the current $2L$-length interval of local propositions (called *mutable*) and move it left-to-right, along with the global propositions. The algorithm searches for a path over these substates and the number of such substates is linear in $n$.

To be able to check the validity of formulas with several local propositions, we must additionally keep track of *recent* local propositions (for which $loc(p) \in \langle i-L, i]$) to the left of the currently *mutable* interval $[i+1, i+2L]$. The truth value of the recent propositions can no longer be changed, but can play a role in validating (or invalidating) formulas where at least one proposition does belong to the mutable interval.

Therefore, the *substate* should contain a truth assignment of an interval of local propositions (from $2L$ mutable and $L$ recent locations) and all global propositions. These propositions of a substate are called *active*. There is at most $(3L+1)M$ active propositions in a given substate. Additionally, to keep track of temporal values (ie, until and eventually), the substate also contains the set of already satisfied such values composed of the active propositions. Let us formalize all of the above notions.

Definition 1. *A* **substate** *is a triple $S = (i, T, \Theta)$ defined by:*

- *the leading index $i \in [0, n-2L]$ of the mutable interval of local propositions,*
- *the set $T$ of currently true active propositions, where active means global, mutable, or recent: $loc(p) = 0$, $loc(p) \in [i+1, i+2L]$, or $loc(p) \in [\max(1, i-L+1), i]$,*
- *the set $\Theta \subseteq \Omega$ of already satisfied formulas of the form $\psi_1 U \psi_2$ or $F\psi$, where $\psi$, $\psi_1$, $\psi_2$ are propositional logic formulas containing only active propositions.*

By multiplying the number of possible choices for each component of $S$, the number of substates is $O(n)$ (for the choice of the leading index) with the large constant factor for the choice of other components, dominated by $O(2^{2^M})$ which is the number of formulas in $\Omega$ containing only active propositions (the maximal cardinality of $\Theta$).

Assume we are given a CONFLICT instance $\mathcal{M} = (\Omega, \gamma, s_o)$ with assumptions from the previous subsection. We will say that a *global* proposition $q$ is *valid* in a substate $S$ if, informally, all values in $\Omega$ containing $q$ are satisfied.

Definition 2. *A global proposition $q$ ($loc(q) = 0$) is* **valid** *in a substate $S = (i, T, \Theta)$ if the following conditions are true for all propositional logic formulas $\psi$ containing $q$:*

- $FG\psi \in \Omega \implies T \models \psi$,
- $F\psi \in \Omega \implies F\psi \in \Theta$,
- $\psi_1 U \psi_2 \in \Omega \implies \psi_1 U \psi_2 \in \Theta$.

Using this definition, the validity of values containing global propositions can be tested in the final substate: the validity of $FG\psi$ formulas depends on this final state ($T$), and for $F\psi$ and $\psi_1 U \psi_2$ formulas we check if they have been satisfied at some previous point ($\Theta$).

On the other hand, the validity of a value containing local propositions is tested when all propositions in the value formula exit the mutable interval, since from that point their truth values cannot be changed. Therefore, we say that a local proposition $p$ is valid in a substate if all values in $\Omega$ containing $p$ and its left (recent) propositions are satisfied.

Definition 3. *A local proposition $p$ ($loc(p) > 0$) is* **valid** *in a substate $S = (i, T, \Theta)$ if the same three conditions as above (Def. 2) are true for all propositional logic formulas $\psi$ such that $\psi$ contains $p$ and every other local proposition $p'$ in $\psi$ is recent ($loc(p') \in [i-L+1, i]$).*

To verify other types of values such as $G\psi$, we posit that a substate must not invalidate any such formula.

Definition 4. *A substate $S = (i, T, \Theta)$ is* **invalid** *if there is a propositional logic formula containing only active propositions such that any of the following is true:*

- $G\psi \in \Omega \quad \wedge \quad T \not\models \psi$,
- $\psi_1 U \psi_2 \in \Omega \quad \wedge \quad \psi_1 U \psi_2 \notin \Theta \quad \wedge \quad T \not\models \psi_1 \quad \wedge \quad T \not\models \psi_2$.

*If a substate is not invalid, we say that it is* **valid**.

The formula $\psi_1 U \psi_2$ becomes satisfied and enters $\Theta$ when $\psi_2 \in T$, assuming that all previous substates were valid (which ensures that $\psi_1$ was true). The formula $F\psi$ enters $\Theta$ when $T \models \psi$.

Definition 5. *Define the* **initial** *substate $S_0$ as the substate $(i, T, \Theta)$ with $i = 0, T = s_0 \cap \{p \in Prop : loc(p) \leq 2L\}$, and $\Theta \subseteq \Omega$ is the set of formulas that are satisfied in the initial state $s_0$, have the form $\psi_1 U \psi_2$ or $F\psi$, and contain only active propositions ($loc(p) \leq 2L$).*

The algorithm that determines if $\mathcal{M}$ is a moral conflict consists in finding an appropriate sequence of substates which simulates a satisfactory action plan. The main idea is to change substates by applying actions that fit the current interval, and move to the next interval when no more actions apply to the left proposition of the interval. Let us formalize the requirements of such a path.

Definition 6. *A directed graph $\mathcal{G}(\mathcal{M})$ (or simply $\mathcal{G}$) has all valid substates as vertices. We define the edges of $\mathcal{G}$ as follows. An edge from substate $S$ to substate $S'$ exists if any of the following two cases ("types" of edges) is satisfied.*

**Type 1:** *There is an action that changes $S$ to $S'$. Formally, there is an index $i$ such that $S = (i, T, \Theta)$ and $S' = (i, T', \Theta')$, and there exists $a \in Act$ such that:*

- $i + 1 \leq l(a) \leq r(a) \leq i + 2L$,
- $T' = T \setminus \{p \in Prop : T \models \gamma^-(a, p),\ loc(p) \in [l(a), r(a)] \cup \{0\}\}$
  $\cup \{p \in Prop : T \models \gamma^+(a, p),\ loc(p) \in [l(a), r(a)] \cup \{0\}\}$,
- *$\Theta'$ is a union of $\Theta$ and the set of all values from $\Omega$ satisfied in $T'$ of the form $\psi_1 U \psi_2$ or $F\psi$, containing only active propositions.*

**Type 2:** *Move right (increase the index). Formally, there is an index $i$ such that $S = (i - 1, T, \Theta)$ and $S' = (i, T', \Theta')$, and*

- $T' = T \setminus \{p \in Prop : loc(p) = i - L\} \cup (s_0 \cap \{p \in Prop : loc(p) = i + 2L\})$,
- *$\Theta'$ is obtained by removing from $\Theta$ the formulas containing propositions $p$ with $loc(p) = i - L$ and adding all values from $\Omega$ satisfied in $T'$ of the form $\psi_1 U \psi_2$ or $F\psi$, containing only active propositions,*
- *all propositions such that $loc(p) = i$ are valid in $S$.*

The last point ensures the validity of the formulas with local propositions that will no longer change their truth value.

Definition 7. *A vertex in $\mathcal{G}$ is* **final** *if it corresponds to a substate $S = (i, T, \Theta)$ such that $i = n - 2L$ (active interval covers the end of Loc) and all propositions with $loc(p) > i$ or $loc(p) = 0$ are valid in $S$.*

Assume that there is a path $\mathcal{P}$ in $\mathcal{G}$ from the initial state $S_0$ to a final state $T$. To generate an action plan $\pi$ that satisfies $\Omega$, we simply take the corresponding actions from all edges of Type 1 from $\mathcal{P}$ in the same order. Therefore, the algorithm for the locality-based CONFLICT problem is given in Algorithm 1 and the following theorem holds.

Theorem 1. *There is a plan $\pi = (a_1, \ldots, a_k)$ such that $H(\pi, s_0, \gamma)$ satisfies $\Omega$ if and only if there is a path in $\mathcal{G}$ from the initial vertex $S_0$ to a final vertex.*

Proof. Algorithm 1 is a depth-first search for any path in $\mathcal{G}$ from the initial vertex $S_0$ to a final vertex. Therefore, if a path exists, Algorithm 1 will find it and produce the corresponding action plan $\pi$. By construction of $\mathcal{G}$, it is clear that $H(\pi, s_0, \gamma)$ satisfies $\Omega$.

For the other direction, assume that $\pi = (a_1, \ldots, a_k)$ is a satisfying action plan. We can construct the corresponding path in $\mathcal{G}$ (sequence of substates) as follows. Start with $t = 1$ and $S_0$ as the initial substate. For each substate index $i = 0, 1, \ldots, n - 2L$, perform the following subprocedure. Apply actions in $\pi$ from $a_t$ onwards (by increasing $t$) such that $r(a_t) \leq i + 2L$, update the current substate according to the action effects (Type-1 edge),

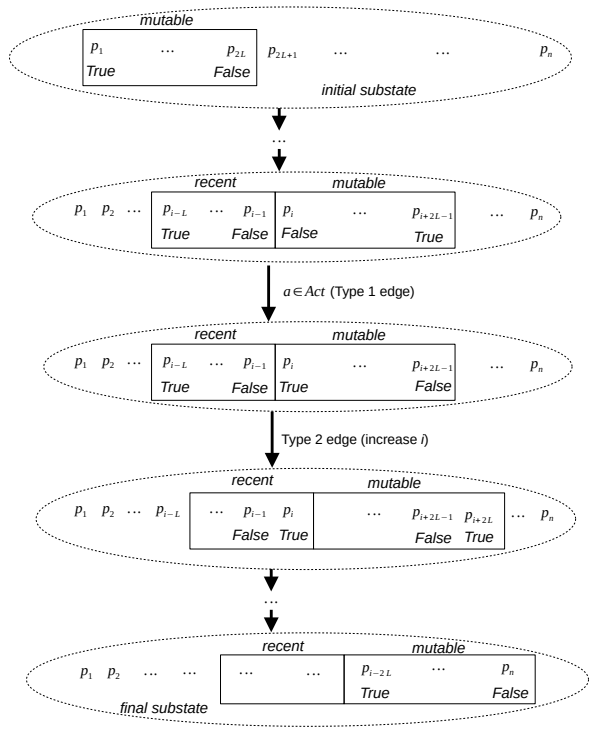


Fig. 1. Illustration of a path in the graph of substates. The index of a proposition represents its location ($l(p_i) = i$)

and finally, when the next action no longer satisfies $r(a_t) \leq i + 2L$, move to the next substate index $i + 1$ (Type-2 edge). Condition (9) ensures that after this subprocedure, no subsequent actions will change propositions with $loc(p) = i + 1$, which means they are valid at that point (the action plan satisfies $\Omega$ by assumption). For the same reason, all propositions when $i = n - 2L$ must be valid. Thus, the state is final. Figure 1 illustrates one potential path in a graph of substates. □

The computational complexity of the locality-based CONFLICT is: $O(n(|Act| + |\Omega|))$ for graph construction and $O(n^2)$ for a breadth-first or depth-first search on a graph of substates. Thus, the locality-based CONFLICT is polynomial.

Because of the large constant factor (simplified to $O(2^{2^M})$ for $K, L = O(M)$), the theoretical strength of this results depends on how small the constants are, which reveals a tradeoff between efficiency and flexibility. However, this worst-case constant will almost never be obtained in practice since the concerned cardinality of $\Theta$ (a local subset of $\Omega$) will usually be far less than the number of all possible propositional logic formulas.

### 3.4 Shortest Satisfactory Plans

Algorithm 1 returns an arbitrary satisfactory plan whenever one exists. A small modification yields a satisfactory plan with the minimum number of actions.

**Algorithm 1** Locality-based CONFLICT

```
Require: Prop, loc, Act, Ω, γ, s_0, L
Ensure: π                                   ▷ Returns a satisfying action plan π or False if it does not exist.
push((S_0, list()))                         ▷ Stack stores (state, action-prefix).
while stack is not empty do
    (S, π) := pop()                         ▷ Take one search node from the stack.
    if visited(S) or not valid(S) then
        continue;
    end if
    visited(S) := True
    if final(S) then return π
    end if
    for a ∈ Act do
        if a changes S to S′ then
            push((S′, π + [a]))             ▷ Type 1 edge appends action.
        end if
    end for
    Construct S′ from S as in Def. 6 (Type 2)
    push((S′, π))
end while
return False
```

Recall that Type 1 edges in Definition 6 correspond to action applications, whereas Type 2 edges only move the active interval to the right. Define the cost of an edge $e$ in $\mathcal{G}$ by

$$c(e) = \begin{cases} 1, & \text{if } e \text{ is of Type 1,} \\ 0, & \text{if } e \text{ is of Type 2.} \end{cases}$$

For a path $\mathcal{P}$ in $\mathcal{G}$, let $c(\mathcal{P})$ be the sum of its edge costs. By construction, $c(\mathcal{P})$ is exactly the number of actions in the extracted plan, since only Type 1 edges contribute actions.

Moreover, every path from the initial vertex $S_0$ to a final vertex contains exactly $n - 2L$ Type 2 edges, because the index $i$ starts at 0, ends at $n - 2L$, and can only be increased by a Type 2 transition. Therefore, among all successful paths, minimizing the total number of edges is equivalent to minimizing the number of Type 1 edges, and hence the number of actions in the resulting plan.

It follows that one may replace the depth-first search in Algorithm 1 by a breadth-first search. Equivalently, one may use a 0–1 shortest-path algorithm with the above edge costs.

Theorem 2. *If there exists a satisfactory plan, then breadth-first search on $\mathcal{G}$ returns a satisfactory plan with the minimum number of actions.*

Proof. Breadth-first search returns a path from $S_0$ to a final vertex with the minimum number of edges. As observed above, every such path contains the same number $n - 2L$ of Type 2 edges. Hence a minimum-edge successful path is also a path with the minimum number of Type 1 edges. Extracting the actions from its Type 1 edges therefore yields a satisfactory plan of minimum length. □

The computational complexity remains polynomial. In particular, replacing depth-first search by breadth-first search does not change the asymptotic bound stated above, although breadth-first search may require more memory in practice.

### 3.5 Best-Effort Optimization and Lexicographic Refinement

When no plan satisfies all values in $\Omega$, one may instead search for a plan that violates values of minimum total weight. Let

$$w : \Omega \to \mathbb{R}_{\geq 0}$$

be a weight function. We then consider a weighted extension of the substate graph in which a transition incurs a penalty whenever some value becomes *irrevocably* unsatisfied on that transition. Informally, local values are settled when their relevant propositions leave the mutable interval, while global values are settled either when they are violated by the current truth assignment or in the final substate.

Thus, the optimization problem is formulated as a shortest-path problem on an augmented substate graph $\mathcal{G}^\star$. The underlying substate still has the form $(i, T, \Theta)$, while the search additionally maintains the bounded status information required to determine which values have already been satisfied or lost. For an edge $e$ of $\mathcal{G}^\star$, let $\lambda(e) \geq 0$ denote the total weight of values that become irrevocably unsatisfied on that transition. For a path $\mathcal{P}$, let

$$\lambda(\mathcal{P}) = \sum_{e \in \mathcal{P}} \lambda(e).$$

A terminal penalty $\tau(S)$ may be added at final vertices to account for values whose truth is determined only in the final substate. Minimizing

$$\lambda(\mathcal{P}) + \tau(S_f)$$

over all paths from the initial vertex to a final vertex $S_f$ is therefore equivalent to maximizing the total satisfied weight.

Since all penalties are nonnegative, this optimization can be solved by a shortest-path algorithm such as Dijkstra's algorithm. However, the graph inherits the layered structure of the original construction: Type 1 edges stay within the same layer $i$, while Type 2 edges move from layer $i-1$ to layer $i$. Therefore, the same computation can be implemented more naturally in a layer-wise manner: for each layer, first compute minimum penalties under Type 1 transitions within that layer, and then propagate these costs through Type 2 transitions to the next layer.

Theorem 3. *Under the same locality assumptions as in the previous subsection, and for fixed $M$, $L$, and $K$, the weighted optimization variant is solvable in polynomial time.*

Proof. Under the locality assumptions, each layer contains only polynomially many distinct augmented substates in the size of the input, with the hidden constant determined by the bounded active window and the bounded status information needed by the soft objective. The number of layers is $O(n)$, and all edge penalties are nonnegative. Hence a shortest-path computation on $\mathcal{G}^\star$ is polynomial. The same conclusion holds for the layer-wise implementation, which performs intra-layer relaxations for Type 1 edges and inter-layer propagation for Type 2 edges. □

A useful refinement is to minimize the number of actions among all best-effort plans of minimum penalty. This is obtained by assigning to each edge the lexicographic cost

$$c^\star(e) = \begin{cases} (\lambda(e), 1), & \text{if } e \text{ is of Type 1,} \\ (\lambda(e), 0), & \text{if } e \text{ is of Type 2,} \end{cases}$$

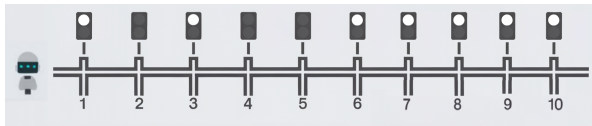


Fig. 2. Example 4.1: Traffic lights 2, 4, and 5 are broken.

and minimizing the total path cost in lexicographic order. The first component minimizes the total lost value weight, while the second component minimizes the number of actions among all value-optimal plans.

## 4 Examples

### 4.1 Traffic Lights

Consider a system that fixes traffic lights along a road. There are $n$ intersections denoted from 1 to $n$ in the order of their position along the road. Let the set of local propositions $Loc = \bigcup_{i=1}^{n}\{on_i, broken_i\}$ denote whether the traffic lights on the corresponding intersections are turned on and whether they are broken (a traffic light can be turned off without being broken). Let the set of global propositions be $Glob = \{congestion\}$ and let $\Omega = \{FGon_i, i = 1, \ldots, n\} \cup \{G\neg congestion\}$, meaning that we want to ultimately fix and turn on all the traffic lights without ever causing congestion. For each intersection, there are three possible actions: $f_i$ fixes and turns on the traffic light, $g_i$ turns it off, and $h_i$ turns it on if it is not broken. Due to synchronicity issues, we can fix the traffic light only if one of the neighboring traffic lights is off. However, if both of them are off while repairing, it causes congestion. Formally,

$$\begin{aligned}
\gamma^-(f_i, broken_i) = \gamma^+(f_i, on_i) = \neg on_{i-1} \vee \neg on_{i+1}, \quad & \text{for } 1 < i < n, \\
\gamma^-(f_i, broken_i) = \gamma^+(f_i, on_i) = \top \quad & \text{for } i \in \{1, n\}, \\
\gamma^+(f_i, congestion) = \neg on_{i-1} \wedge \neg on_{i+1} \quad & \text{for } 1 < i < n, \\
\gamma^-(g_i, on_i) = \top, \quad \gamma^+(h_i, on_i) = \neg broken_i, \quad & \text{for } 1 \leq i \leq n. \qquad (10)
\end{aligned}$$

$\mathcal{M} = (\Omega, \gamma, s_0)$ is a locality-based moral conflict for $M = |Glob| = 1$ and $L = 3$ since each action concerns at most three consecutive local propositions. $L = 3$ also means that actions $f_i$, $g_i$ and $h_i$ can be performed after $f_j$, $g_j$ and $h_j$ for $j \leq i + 3$, but not after $f_{i+4}$, $g_{i+4}$ and $h_{i+4}$. Thus, the order of fixing traffic lights is roughly similar to their order along the road, which is a natural constraint.

For $n = 10$ and the initial state $s_0 = \{on_1, broken_2, on_3, broken_4, broken_5, on_6, on_7, on_8, on_9, on_{10}\}$ as depicted in Figure 2, $\mathcal{M}$ is not a moral conflict. There is an action plan $\pi = (g_3, f_2, h_3, f_4, g_6, f_5, h_6)$. Namely, $g_3$ turns off light 3 so we can fix light 2 using $f_2$; then we turn on light 3 again using $h_3$. We can then fix light 4 using $f_4$. If we used $f_4$ before $h_3$ (ie, before light 3 was turned on), we would cause congestion. To fix light 5, we turn off light 6 using $g_6$ and then perform $f_5$ before turning light 6 on again using $h_6$. For this example, the proposed algorithm finds the following path, ie, the sequence of substates:

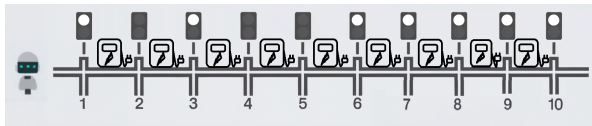


Fig. 3. Example 4.2: Traffic lights 2, 4, and 5 are broken, and there is a charger between each pair of lights. Fixing traffic light 4 drains the robot's battery.

$$
\begin{aligned}
& (0,\ \{\{on_1, broken_2, on_3, broken_4, broken_5, on_6\},\ \emptyset\}, \emptyset) \\
\text{perform } g_3 \ &\rightarrow (0,\ \{\{on_1, broken_2, broken_4, broken_5, on_6\},\ \emptyset\}, \emptyset) \\
\text{perform } f_2 \ &\rightarrow (0,\ \{\{on_1, on_2, broken_4, broken_5, on_6\},\ \emptyset\}, \emptyset) \\
\text{perform } h_3 \ &\rightarrow (0,\ \{\{on_1, on_2, on_3, broken_4, broken_5, on_6\},\ \emptyset\}, \emptyset) \\
\text{move} \ &\rightarrow (1,\ \{\{on_1, on_2, on_3, broken_4, broken_5, on_6, on_7\},\ \emptyset\}, \emptyset) \\
\text{perform } f_4 \ &\rightarrow (1,\ \{\{on_1, on_2, on_3, on_4, broken_5, on_6, on_7\},\ \emptyset\}, \emptyset) \\
\text{move} \ &\rightarrow (2,\ \{\{on_1, on_2, on_3, on_4, broken_5, on_6, on_7, on_8\},\ \emptyset\}, \emptyset) \\
\text{perform } g_6 \ &\rightarrow (2,\ \{\{on_1, on_2, on_3, on_4, broken_5, on_7, on_8\},\ \emptyset\}, \emptyset) \\
\text{perform } f_5 \ &\rightarrow (2,\ \{\{on_1, on_2, on_3, on_4, on_5, on_7, on_8\},\ \emptyset\}, \emptyset) \\
\text{perform } h_6 \ &\rightarrow (2,\ \{\{on_1, on_2, on_3, on_4, on_5, on_6, on_7, on_8\}, \emptyset\}, \emptyset) \\
\text{move} \ &\rightarrow (3,\ \{\{on_1, on_2, on_3, on_4, on_5, on_6, on_7, on_8, on_9\},\ \emptyset\}, \emptyset) \\
\text{move} \ &\rightarrow (4,\ \{\{on_2, on_3, on_4, on_5, on_6, on_7, on_8, on_9, on_{10}\},\ \emptyset\}, \emptyset).
\end{aligned}
$$

## 4.2 Traffic Lights with Chargers

As depicted in Figure 3, we expand the previous example by introducing charging stations between each pair of traffic lights. The charging stations allow the robot to recharge its battery. We model a maintenance policy in which one major repair drains the battery and exactly one recharge is expected afterward. Charging before the battery is low should be disallowed, and repeated charging after recovery should also be disallowed. We introduce an action $c_i$ for $i = 1, \ldots, n-1$ that recharges the robot at the station between traffic lights $i$ and $i+1$. Proposition $charged_i$ records which station was used, and global proposition $chargedOnce$ records whether a recharge has already happened.
Formally, we have:

$$
\begin{aligned}
Loc &= \bigcup_{i=1}^{n} \{on_i,\ broken_i\} \ \cup \ \bigcup_{i=1}^{n-1} \{charged_i\}, \quad Glob = \{congestion, lowBattery, chargedOnce\}, \\
\Omega &= \{FGon_i,\ i = 1, \ldots, n\} \ \cup \ \{\neg charged_i \ U \ lowBattery,\ i = 1, \ldots, n-1\} \\
&\quad \cup \ \{FGchargedOnce\} \ \cup \ \{G\neg congestion\} \ \cup \ \{FG\neg lowBattery\}.
\end{aligned} \tag{11}
$$

The initial state is the same as in Example 4.1, extended with battery-related propositions: $lowBattery$ is false, $charged_i$ is false for $i = 1, \ldots, n-1$, and $chargedOnce$ is false. We update the actions so that, for $i = 1, \ldots, n-1$,

$$
\gamma^-(c_i, lowBattery) = \gamma^+(c_i, charged_i) = \gamma^+(c_i, chargedOnce) = lowBattery \wedge \neg chargedOnce. \tag{12}
$$

Thus, charging has an effect only when the battery is low and no prior charging event has occurred. Major repair work will include fixing traffic light 4, thus $\gamma^+(f_4, lowBattery) = \top$. Actions $f_i, g_i, h_i$ can be performed only if the

battery is not low:

$$\begin{aligned}
\gamma^-(f_i, broken_i) = \gamma^+(f_i, on_i) &= (\neg on_{i-1} \vee \neg on_{i+1}) \wedge \neg lowBattery && \text{for } 1 < i < n, \\
\gamma^-(f_i, broken_i) = \gamma^+(f_i, on_i) &= \neg lowBattery && \text{for } i \in \{1, n\}, \\
\gamma^-(g_i, on_i) = \neg lowBattery, \quad \gamma^+(h_i, on_i) &= \neg broken_i \wedge \neg lowBattery && \text{for } 1 \leq i \leq n.
\end{aligned} \tag{13}$$

Again, since each action affects at most three consecutive local propositions, we have $L = 3$. The algorithm finds the following path, ie, the sequence of substates:

$$\begin{aligned}
& (0, \{\{on_1, broken_2, on_3, broken_4, broken_5, on_6\}, \emptyset\}, \emptyset) \\
\text{perform } g_3 &\rightarrow (0, \{\{on_1, broken_2, broken_4, broken_5, on_6\}, \emptyset\}, \emptyset) \\
\text{perform } f_2 &\rightarrow (0, \{\{on_1, on_2, broken_4, broken_5, on_6\}, \emptyset\}, \emptyset) \\
\text{perform } h_3 &\rightarrow (0, \{\{on_1, on_2, on_3, broken_4, broken_5, on_6\}, \emptyset\}, \emptyset) \\
\text{move} &\rightarrow (1, \{\{on_1, on_2, on_3, broken_4, broken_5, on_6, on_7\}, \emptyset\}, \emptyset) \\
\text{perform } f_4 &\rightarrow (1, \{\{on_1, on_2, on_3, on_4, broken_5, on_6, on_7\}, \{lowBattery\}\}, \cup_{i=1}^{7}\{\neg charged_i \ U \ lowBattery\}) \\
\text{move} &\rightarrow (2, \{\{on_1, on_2, on_3, on_4, broken_5, on_6, on_7, on_8\}, \{lowBattery\}\}, \cup_{i=1}^{8}\{\neg charged_i \ U \ lowBattery\}) \\
\text{perform } c_4 &\rightarrow (2, \{\{on_1, on_2, on_3, on_4, charged_4, broken_5, on_6, on_7, on_8\}, \{chargedOnce\}\}, \cup_{i=1}^{8}\{\neg charged_i \ U \ lowBattery\}) \\
\text{move} &\rightarrow (3, \{\{on_1, on_2, on_3, on_4, broken_5, on_6, on_7, on_8, on_9\}, \{chargedOnce\}\}, \cup_{i=1}^{9}\{\neg charged_i \ U \ lowBattery\}) \\
\text{perform } g_6 &\rightarrow (3, \{\{on_1, on_2, on_3, on_4, broken_5, on_7, on_8, on_9\}, \{chargedOnce\}\}, \cup_{i=1}^{9}\{\neg charged_i \ U \ lowBattery\}) \\
\text{perform } f_5 &\rightarrow (3, \{\{on_1, on_2, on_3, on_4, on_5, on_7, on_8, on_9\}, \{chargedOnce\}\}, \cup_{i=1}^{9}\{\neg charged_i \ U \ lowBattery\}) \\
\text{perform } h_6 &\rightarrow (3, \{\{on_1, on_2, on_3, on_4, on_5, on_6, on_7, on_8, on_9\}, \{chargedOnce\}\}, \cup_{i=1}^{9}\{\neg charged_i \ U \ lowBattery\}) \\
\text{move} &\rightarrow (4, \{\{on_2, on_3, on_4, on_5, on_6, on_7, on_8, on_9, on_{10}\}, \{chargedOnce\}\}, \cup_{i=2}^{9}\{\neg charged_i \ U \ lowBattery\})
\end{aligned}$$

The last substate is indeed final since $i = n - 2L$ and all active propositions are valid in the sense of Definitions 2 and 3. For example, $FGon_7$, $FG\neg lowBattery$, and $FGchargedOnce$ are true, while $\neg charged_7 \ U \ lowBattery$ does belong to $\Theta$. The validity of previous local propositions, such as $on_1$, has been verified as part of the "move" operation according to Definition 6, while the truth of $G\neg congestion$ was verified in each substate according to Definition 4.

### 4.3 Narrow Factory Grid (2D)

The previous examples are effectively one-dimensional: intersections are ordered along a road. We now show a two-dimensional setting that still satisfies the locality assumptions when one dimension is small. Consider a long production line with $n$ columns and $W$ parallel conveyor lanes, where $W$ is a fixed small constant (eg, $W = 3$). The robot can move forward along the line and switch between neighboring lanes, while human workers may temporarily occupy specific crossing cells that must be avoided.

Figure 4 illustrates the concrete instance used in this subsection. The robot starts in the middle lane at column 1. Required service cells are in the middle lane at columns 3, 7, and 9, while human-occupied cells appear in the middle lane at columns 4 and 8. A direct middle-lane traversal would enter those occupied cells and violate safety. Therefore, after repairing column 3, the robot detours to another lane before column 4 and returns to the middle lane afterward; later it performs an analogous detour around column 8 before returning to repair column 9. After completing the required repairs, it must continue to the terminal target cell at column 12 in the middle lane.

We index each cell by $(i, y)$ where $i \in \{1, \ldots, n\}$ is the column and $y \in \{1, \ldots, W\}$ is the lane. The key modeling choice is to map locality to columns, ie, we define $loc(\cdot) = i$ for every proposition indexed by $(i, y)$. This keeps the

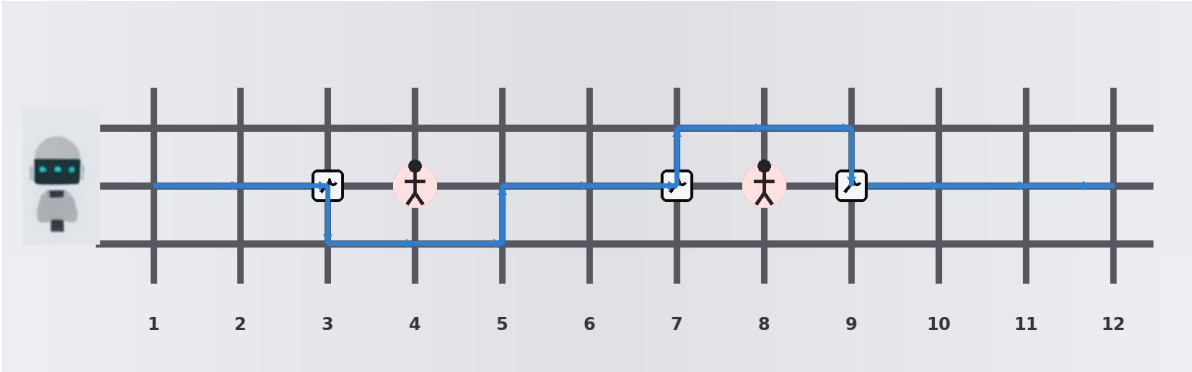


Fig. 4. Example 4.3: A three-lane production corridor with required service cells at columns 3, 7, and 9 (middle lane), occupied crossing cells at columns 4 and 8, and terminal target at column 12 in the middle lane. The highlighted route shows a collision-free detour using lane changes and continuation to the terminal cell.

"long" direction explicit, while the lane index is absorbed into the bounded number of propositions per location. Formally,

$$Loc = \bigcup_{i=1}^{n} \{robot_{i,y}, done_{i,y}, human_{i,y} : y = 1, \dots, W\}, \qquad Glob = \{collision\}. \tag{14}$$

Hence each location $i$ has at most $3W$ local propositions, so $M = 3W$ is constant when $W$ is fixed.

Let $R \subseteq \{1, \dots, n\} \times \{1, \dots, W\}$ be the set of stations that must be serviced. The value set is

$$\Omega = \{G\neg collision\} \ \cup \ \{FG\, done_{i,y} : (i, y) \in R\} \ \cup \ \{FG\, robot_{n,2}\}. \tag{15}$$

Thus, collisions are always forbidden, every required station must eventually be repaired and remain repaired, and the robot must eventually reach and remain at the terminal middle-lane cell $(n, 2)$.

We use three action families:

- *Forward move* $m_{i,y}$ for $1 \le i < n$: move from $(i, y)$ to $(i + 1, y)$.
- *Lane change* $s_{i,y\to y'}$ for $|y - y'| = 1$: move at fixed column $i$ from lane $y$ to lane $y'$.
- *Repair* $rep_{i,y}$: repair station at $(i, y)$.

Representative effects are:

$$\begin{aligned} \gamma^-(m_{i,y}, robot_{i,y}) &= robot_{i,y}, \\ \gamma^+(m_{i,y}, robot_{i+1,y}) &= robot_{i,y}, \\ \gamma^+(m_{i,y}, collision) &= robot_{i,y} \wedge human_{i+1,y}. \end{aligned} \tag{16}$$

$$\begin{aligned} \gamma^-(s_{i,y\to y'}, robot_{i,y}) &= robot_{i,y}, \\ \gamma^+(s_{i,y\to y'}, robot_{i,y'}) &= robot_{i,y}, \\ \gamma^+(s_{i,y\to y'}, collision) &= robot_{i,y} \wedge human_{i,y'}. \end{aligned} \tag{17}$$

$$\begin{aligned} \gamma^+(rep_{i,y}, done_{i,y}) &= robot_{i,y} \wedge \neg human_{i,y}, \\ \gamma^+(rep_{i,y}, collision) &= robot_{i,y} \wedge human_{i,y}. \end{aligned} \tag{18}$$

For the concrete three-lane encoding $(A, B, C)$, these schemas ground to the input rules. In particular, collision updates include

$$\begin{aligned}
&\gamma^+(mA_i, collision) = ra_i \wedge ha_{i+1}, \quad \gamma^+(mB_i, collision) = rb_i \wedge hb_{i+1}, \quad \gamma^+(mC_i, collision) = rc_i \wedge hc_{i+1}, \\
&\gamma^+(sAB_i, collision) = ra_i \wedge hb_i, \quad \gamma^+(sBA_i, collision) = rb_i \wedge ha_i, \quad \gamma^+(sBC_i, collision) = rb_i \wedge hc_i, \\
&\gamma^+(sCB_i, collision) = rc_i \wedge hb_i.
\end{aligned} \tag{19}$$

Likewise, movement effects are source-conditioned (e.g., $\gamma^-(mA_i, ra_i) = ra_i$ and $\gamma^+(mA_i, ra_{i+1}) = ra_i$), which exactly matches rules such as "`+ mA1 collision : ra1 AND ha2`" in the instantiated specification. All other effects are $\perp$. In this model, move actions touch only columns $i$ and $i+1$, lane changes and repairs touch only column $i$, and all of them may touch one global proposition (*collision*). Therefore $r(a) - l(a) \leq 1$ for all actions, so the locality requirement is satisfied for $L = 2$.

For the concrete instance from Figure 4, let $n = 12$, $W = 3$, $R = \{(3, 2), (7, 2), (9, 2)\}$, and let the initial robot position be $(1, 2)$. Crossing zones $(4, 2)$ and $(8, 2)$ are occupied by humans. A satisfying plan is:

$$\begin{aligned}
\pi = (&m_{1,2}, m_{2,2}, rep_{3,2}, s_{3,2\to3}, m_{3,3}, m_{4,3}, s_{5,3\to2}, \\
&m_{5,2}, m_{6,2}, rep_{7,2}, s_{7,2\to1}, m_{7,1}, m_{8,1}, s_{9,1\to2}, rep_{9,2}, \\
&m_{9,2}, m_{10,2}, m_{11,2}).
\end{aligned}$$

The robot repairs required cells, detours around occupied crossing zones by switching lanes before entering unsafe cells, and then continues to the terminal target $B12$. This example is two-dimensional in geometry, but remains one-dimensional in locality indexing with a fixed-width augmentation, which is exactly the regime targeted by our complexity assumptions.

## 5 Evaluation

This section evaluates whether the theoretical locality advantage translates into practical runtime gains. We compare three planners under a common protocol: the proposed locality-based planner, a brute-force planner with online value pruning, and an automata/progression $LTL_f$ baseline. The automata/progression baseline follows the standard finite-trace temporal-logic approach in which search is performed on a product between planning states and temporal-monitor states (**degiacomo2013ltlf**; **camacho2018finite**); operationally, each value formula is represented by a residual formula that is progressed after each action. In all experiments, every produced plan is checked by an independent validator, so correctness is not inferred from internal planner state.

### 5.1 Setup

All experiments were run on an Intel Core i7-10510U (4 cores / 8 threads) machine with 31 GiB RAM, Ubuntu Linux kernel 5.15, and C++17 implementations compiled with `g++ 9.4.0` using optimization level `-O2`. Code, benchmark-generation scripts, and experiment artifacts are available at https://github.com/satja/LTL.

The benchmark set consists of 110 systematically generated instances with problem size $n \in [6, 16]$ and ten instances per size. The generation procedure is deterministic and size-ordered, with nontrivial pattern variation per size: damaged-location patterns in Example 1 and Example 2, and service/occupied-cell patterns in Example 3. Each family is generated from the same action/value definitions used in its corresponding example subsection, including the one-charge policy in Example 2 and the source-conditioned lane/move effects in Example 3. Family assignment follows a fixed cyclic pattern over variants, yielding 44 instances from the linear traffic-light family (Example 1), 33 from the charging extension (Example 2), and 33 from the three-lane repair corridor (Example 3). This design provides controlled growth in instance size while preserving the locality assumptions studied in Section 3.2, while also testing whether conclusions remain stable across all three modeling variants rather than only one. In the plots, this means that for each fixed $n$ there are exactly ten benchmark instances.

All methods are run with the same maximum search depth (160) and evaluated per instance under equal fixed runtime limits of 15 seconds per planner. The locality parameter is fixed at $L = 3$, matching the largest locality width among the benchmark families. Example 4.3 itself already satisfies locality for $L = 2$, but we keep a single fixed $L$ across all families for direct comparability. Although the formal framework does not impose a global bound on plan length, this depth cap is used in experiments for fair and controlled comparison, so empirical completeness claims are with respect to that cap. A run is counted as successful only if a planner returns a plan within the time limit and that plan is accepted by independent validation. Reported planning times exclude validation time, so runtime values represent planner-only cost.

## 5.2 Results

Table 1 summarizes success counts and runtime statistics on the full 110-instance suite. The locality-based planner solves all 110 instances within the 15-second limit, and all 110 successful outputs are validated. The automata/progression baseline solves 105 instances and times out on 5, with all 105 successful outputs validated. The brute-force baseline solves 90 instances and times out on 20, with all 90 successful outputs validated. Runtime statistics are computed over successful runs only: the locality planner has mean runtime 0.018s, median 0.00s, and maximum 0.12s; the automata/progression baseline has mean 0.572s, median 0.03s, and maximum 12.05s; and the brute-force baseline has mean 1.576s, median 0.25s, and maximum 13.81s.

To test whether the runtime advantage is statistically significant at the distribution level, we apply one-sided Mann–Whitney U tests with alternative hypothesis that locality runtimes are smaller than baseline runtimes. Using per-instance planner times (including timeout-capped values), we obtain $p = 1.86 \times 10^{-31}$ for locality versus brute force and $p = 3.90 \times 10^{-14}$ for locality versus automata/progression. Under this equal-timeout setting, locality is significantly faster than both baselines in the capped per-instance distribution. Because all three methods use the same 15-second timeout cap, the rank-based comparison does not depend on asymmetric censoring thresholds.

Table 1. Benchmark summary over 110 instances (successful runs only for runtime statistics).

| Method | Success | Timeout | Mean (s) | Median (s) | Max (s) |
|---|---|---|---|---|---|
| Locality-based planner | 110/110 | 0 | 0.018 | 0.00 | 0.12 |
| Automata/progression baseline | 105/110 | 5 | 0.572 | 0.03 | 12.05 |
| Brute-force baseline | 90/110 | 20 | 1.576 | 0.25 | 13.81 |

To make the cross-example effect explicit, Table 2 reports success counts separately for the three benchmark families. The locality planner remains complete in each family under the common 15-second cap. The automata/progression baseline is complete on Example 1 and Example 3, and loses robustness on Example 2. Brute force times out on a subset of Example 1 and on roughly half of Example 2, while remaining complete on Example 3 in this suite.

Table 2. Success counts by benchmark family (timeout limit: 15s for all methods).

| Family | Locality-based | Automata/progression | Brute-force |
|---|---|---|---|
| Example 1 (linear traffic lights, 44 instances) | 44/44 | 44/44 | 41/44 |
| Example 2 (charging extension, 33 instances) | 33/33 | 28/33 | 16/33 |
| Example 3 (three-lane corridor, 33 instances) | 33/33 | 33/33 | 33/33 |

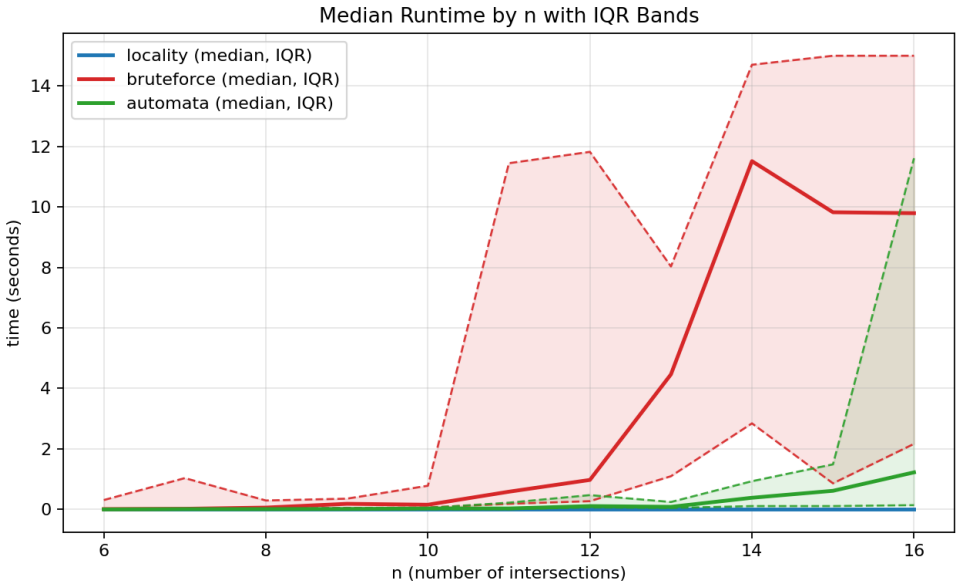


Fig. 5. Runtime versus instance size $n$ with ten instances per size. Solid lines show median runtime for each method and dashed side lines show the first and third quartiles (IQR band).

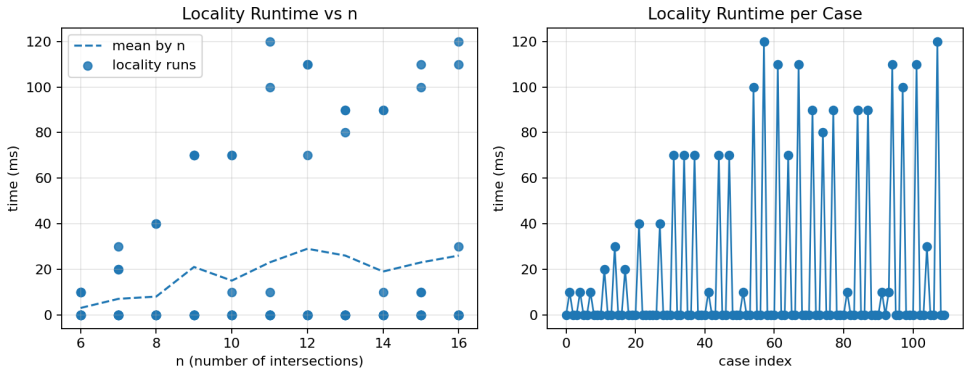


Fig. 6. Locality-only runtime detail. The left panel shows all locality runs grouped by $n$ with the mean trend, and the right panel shows the same runs by case index. Isolating this method reveals its internal growth pattern that is visually compressed in the multi-method comparison.

## 5.3 Results Discussion

The central empirical observation is that locality-aware state compression dominates in this benchmark regime. Under the fixed setting $L = 3$ and a common 15-second budget, the locality planner solves all instances and has the lowest runtime distribution. This matters because the three families stress different parts of the model: the first

emphasizes local repair propagation with global congestion safety, the second adds charging-related propositions and additional temporal obligations, and the third introduces lane switches and collision-avoidance structure in a narrow two-dimensional geometry. Figure 5 provides the aggregate view by number of intersections using robust summary statistics: for each method, the median runtime grows with $n$, and the dashed bands show the interquartile range. Figure 6 complements this by isolating only the locality planner. This view confirms that locality runtime remains low across the full size range, including larger instances in the suite.

The baseline behavior is heterogeneous by family, which helps explain aggregate statistics. On Example 1, locality and automata/progression are both complete, while brute force loses robustness on a small subset. On Example 2, locality is complete, automata/progression times out on a nontrivial subset, and brute force times out on roughly half of instances. On Example 3, all three methods are complete under the 15-second budget, but locality still maintains the lowest runtime distribution.

These results are consistent with the theory from Section 3.3. The proposed method keeps only a bounded active window of local propositions, together with global propositions and compact temporal-status memory, and shifts this window left-to-right. In this locality-structured benchmark, that representation yields both strong pruning and low per-state overhead, which explains the observed runtime and completion advantage. The automata/progression baseline, by construction, explores product states that combine planning valuations with residual temporal formulas, and this temporal-monitor overhead is most visible in the charger family where temporal obligations are denser. The brute-force baseline suffers most from direct state-space growth, despite online pruning.

At the same time, interpretation should remain scoped to the evaluated regime. All three families are locality-structured by design and therefore aligned with the assumptions of our algorithm. This is appropriate for validating the intended use case and stronger than evaluating only one family, but it is still not evidence that the same margins will hold on domains with weak locality or substantially different temporal-value distributions.

## 6 Conclusion

We introduced a locality-based approach to solving the CONFLICT problem in ethical planning using Linear Temporal Logic (LTL). By incorporating assumptions on the structure of the problem, the distinction between global and local propositions and the constraints on the scope and sequence of actions, we were able to reduce the complexity of the problem from PSPACE-complete to polynomial time. The introduced concept of localized planning restricts the actions to only affect and depend on propositions within a bounded interval. It incorporates constraints on the temporal order of actions based on this interval. Due to possibility of large constant factors in the algorithm complexity, there might be a tradeoff between efficiency and flexibility. We consider the proposed algorithm to be practical for real-world applications with small values of constants $K$, $L$, and $M$, ie, the applications with "high locality". The evaluation in Section 5 supports this claim by showing both runtime and robustness advantages on the systematic locality-structured benchmark suite.

Overall, our work offers a promising direction for efficient and scalable ethical planning in autonomous systems and robotics. By constraining the problem space in a principled way, we enable the development of action plans that adhere to predefined values, ensuring ethical behavior in complex, dynamic environments. Future research could explore extending this framework to accommodate more complex moral structures and larger sets of global propositions, as well as applying these methods to real-world robotic systems and other automated decision-making contexts.

## A Reproducibility Checklist for JAIR

Select the answers that apply to this research.

## All articles

(1) All claims investigated in this work are clearly stated. [yes]
(2) Clear explanations are given how the work reported substantiates the claims. [yes]
(3) Limitations or technical assumptions are stated clearly and explicitly. [yes]
(4) Conceptual outlines and/or pseudo-code descriptions of the AI methods introduced in this work are provided, and important implementation details are discussed. [yes]
(5) Motivation is provided for all design choices, including algorithms, implementation choices, parameters, data sets and experimental protocols beyond metrics. [yes]

## Articles containing theoretical contributions

Does this paper make theoretical contributions? [yes]

(1) All assumptions and restrictions are stated clearly and formally. [yes]
(2) All novel claims are stated formally (e.g., in theorem statements). [yes]
(3) Proofs of all non-trivial claims are provided in sufficient detail to permit verification by readers with a reasonable degree of expertise. [yes]
(4) Complex formalism, such as definitions or proofs, is motivated and explained clearly. [yes]
(5) The use of mathematical notation and formalism serves the purpose of enhancing clarity and precision; gratuitous use of mathematical formalism is avoided. [yes]
(6) Appropriate citations are given for all non-trivial theoretical tools and techniques. [yes]

## Articles reporting on computational experiments

Does this paper include computational experiments? [yes]

(1) All source code required for conducting experiments is included in an online appendix or will be made publicly available upon publication of the paper. [yes]
(2) The source code comes with a license that allows free usage for reproducibility purposes. [yes]
(3) The source code comes with a license that allows free usage for research purposes in general. [yes]
(4) Raw, unaggregated data from all experiments is included in an online appendix or will be made publicly available upon publication of the paper. [yes]
(5) The unaggregated data comes with a license that allows free usage for reproducibility purposes. [yes]
(6) The unaggregated data comes with a license that allows free usage for research purposes in general. [yes]
(7) If an algorithm depends on randomness, then the method used for generating random numbers and for setting seeds is described in a way sufficient to allow replication of results. [yes]
(8) The execution environment for experiments, including hardware and software stack, is described. [yes]
(9) The evaluation metrics used in experiments are clearly explained and their choice is explicitly motivated. [yes]
(10) The number of algorithm runs used to compute each result is reported. [yes]
(11) Reported results have not been "cherry-picked" by silently ignoring unsuccessful or unsatisfactory experiments. [yes]
(12) Analysis of results goes beyond single-dimensional summaries of performance to include measures of variation, confidence, or other distributional information. [yes]
(13) All (hyper-) parameter settings for the algorithms/methods used in experiments have been reported, along with the rationale or method for determining them. [yes]
(14) The number and range of (hyper-) parameter settings explored prior to conducting final experiments have been indicated, along with the effort spent on (hyper-) parameter optimisation. [yes]

(15) Appropriately chosen statistical hypothesis tests are used to establish statistical significance in the presence of noise effects. [yes]

## Articles using data sets

Does this work rely on one or more data sets (possibly obtained from a benchmark generator or similar software artifact)? [yes]

(1) All newly introduced data sets are included in an online appendix or will be made publicly available upon publication of the paper. [yes]
(2) The newly introduced data set comes with a license that allows free usage for reproducibility purposes. [yes]
(3) The newly introduced data set comes with a license that allows free usage for research purposes in general. [yes]
(4) All data sets drawn from the literature or other public sources are accompanied by appropriate citations. [NA]
(5) All data sets drawn from the existing literature are publicly available. [NA]
(6) All new data sets and data sets that are not publicly available are described in detail, including relevant statistics and data collection process if relevant. [yes]
(7) All methods used for preprocessing, augmenting, batching or splitting data sets are described in detail. [NA]

## Explanations on answers above (optional)

Code and benchmark artifacts are available in the project repository and released under the MIT License.

Received ; accepted